

PAMOJABUILD PRODUCTION SPECIFICATION

MULTI-TENANT ESCROW, WEB-CRYPTO CO-SIGNING & LIGHTNING BACKEND GUIDE

1. Introduction & Core Problem Space

This specification establishes the production-grade architecture for **PamojaBuild**, a multi-tenant donation platform designed to enable trust-minimized, regionalized financial workflows. The application facilitates high-frequency, low-fee micro-donations via the Lightning Network on Layer 2, while securing funds within isolated multi-signature cold storage escrows on Layer 1.

The Multitenancy Constraint: Every donation task initialized on the platform maintains an completely independent set of five (5) local community Trustees (e.g., distinct community councils for Kibera versus Mamboleo). Therefore, a fundamental requirement is that funds must remain cryptographically segregated per task, preventing any cross-tenant accessibility or pooled vulnerability.

2. Architectural Decisions & Trade-Off Matrix

Building multi-tenant applications on top of the Lightning Network presents unique routing and security constraints. Below is the technical rationale behind the design choices selected for PamojaBuild compared to alternative models evaluated.

2.1 Network Topology & Inbound Liquidity Management

Approach Evaluated	Why It Fails in Production / Constraints	PamojaBuild Selected Verdict & Rationale
One Channel Per Task (Network Isolation)	Liquidity Fragmentation: Capital must be locked in individual channels with an LSP for every active task. If one task goes viral, its channel jams once filled, while idle channels trap paid liquidity.	Single Mega-Node with Shared Channel Pools + Off-Chain Ledger Management Maximizes capital efficiency. All incoming donations hit a single high-capacity inbound pipeline. Sifting and allocation are handled programmatically by the backend database layer, ensuring no donor payment ever fails due to empty channel bottlenecks.
One LND Node Per Task (Process Isolation)	DevOps & Fee Bleed: Drastically increases operational overhead. Opening and closing dedicated lightning channels for hundreds of micro-tasks bleeds substantial capital via Layer 1 base fees.	

2.2 Small Task and Tail Balance Handling

When a donation task ends or remains small, funds sit as a "tail balance" inside active Lightning channels. If the volume is too small to justify an on-chain Submarine Swap, an alternative settlement route must be carved out.

Approach Evaluated	Why It Fails in Production / Constraints	PamojaBuild Selected Verdict & Rationale
Forced Micro-Swap (Push completely to L1)	Fee Cannibalization: Forcing a 50,000 Satoshi (\$30) tail balance onto Layer 1 causes base-chain miner fees to consume a significant percentage of the project value, shortchanging the volunteer.	Web-Cryptographic Co-Signing & Dual-Layer Split Payout (Solution A) Maintains pure capital efficiency. Large blocks are swept to on-chain 3-of-5 scripts, while small/tail balances remain on Layer 2. Security is guaranteed because LND cannot release funds without verifying the cryptographic browser signatures of 3 out of 5 task-specific trustees.
Database Logic Only (App-side approvals)	Centralized Honey Pot: Relying on a standard SQL boolean field (e.g., <code>is_approved = true</code>) means a database injection or web-server compromise completely strips control from local trustees, leading to immediate hot wallet draining.	

3. End-to-End Core Lifecycle

1. **Task Setup:** 5 regional trustees register their Extended Public Keys (Xpubs) and browser-side Web-Crypto public keys to a unique Task container.
2. **Ingestion:** Invoices are generated dynamically by LND. Settled balances are mapped to the task via internal ledgers secured by HMAC signatures.
3. **The Flush (Large Volumes):** When a task's unswapped ledger hits a threshold (e.g., 1,000,000 Sats), a Submarine Swap is automatically fired, draining L2 liquidity into a newly derived 3-of-5 on-chain address for that task.
4. **The Payout (Tail/Small Balances):** When the project concludes, the volunteer provides an on-chain address and a Lightning invoice. The trustees execute a single unified approval action, simultaneously releasing the L1 vault funds and the L2 tail balance securely.

4. Detailed Backend Implementation Blueprint

This blueprint maps the implementation specifications required by developers and autonomous coding agents using the Go programming language ecosystem.

4.1 Open-Source Library Dependency Mapping

- `github.com/btcsuite/btcd/btcutil/hdkeychain` : Parses task Xpub strings and performs deterministic child key derivations for multi-signature addresses.

- `github.com/btcsuite/btcd/txscript` : Builds the actual 3-of-5 multi-signature script opcodes used to secure Layer 1 vaults.
- `github.com/lightningnetwork/lnd/lnrpc` : Provides native gRPC/REST clients to command the LND daemon (`CreateInvoice` , `SendPayment` , `PublishTransaction`).
- `crypto/hmac` (Standard Library): Constructs tamper-proof cryptographic audit trails across database rows.
- `crypto/ed25519` (Standard Library): Validates browser-side trustee signatures during Layer 2 web-crypto co-signing evaluations.

4.2 The Relational Task State Machine

To prevent race conditions, double-spending, or trapped funds, every task must progress through a strict, linear database state machine:

State Constant	Database Integrity / Business Logic Rules	Allowed Transitions
<code>ACTIVE</code>	Donations are actively accepted. LND invoices can be bound to this Task ID. Automated swaps execute when the threshold is violated.	<code>LIQUIDATING</code>
<code>LIQUIDATING</code>	New donations are blocked. The backend locks the task ledger and performs final assessment of any remaining Layer 2 tail balances.	<code>READY_FOR_PAYOUT</code>
<code>READY_FOR_PAYOUT</code>	The final balance sheet is locked. The backend exposes raw transaction structures to the frontend for trustee cryptographic voting.	<code>PAYOUT_PROCESSING</code>
<code>ARCHIVED</code>	Vault balances are zeroed. Volunteers are verified paid. Task cryptographic records are retired to cold storage.	None (Terminal State)

4.3 Cryptographic Audit Trail (Anti-Tampering Engine)

To neutralize the threat of a database injection or an unauthorized administrator modifying task rows, the Go backend must chain transactions together using an HMAC signature.

Mathematical Formula for Row Integrity

$$HMAC_Signature = HMAC_SHA256(Secret_Key, Task_ID + String(Amount) + Previous_Row_Hash + String(Timestamp))$$

Prior to executing any Submarine Swap or initiating a payout sequence, the Go service must query the task's transaction ledger history sequentially, recalculating the HMAC chain from the genesis record. If a mismatch is discovered, the backend automatically flags a severe system breach, freezes the LND webhook handlers, and aborts the pipeline.

5. Frontend Integration Contract & API Touchpoints

This section establishes the explicit interface definitions and data schemas required by the frontend engineering team to bridge user interactions with our backend cryptographic engine.

5.1 Donor Ingestion: Fetching a Payment Request

Endpoint: `POST /api/v1/tasks/:task_id/donate`

Payload: `{"amount_sats": 25000}`

Response Schema:

```
{
  "task_id": "kibera-clean-water-004",
  "invoice_bolt11": "lnbc250u1p398z...",
  "payment_hash_hex": "4a7d1ed26a8c...",
  "expires_at": "2026-06-17T23:55:00Z"
}
```

5.2 The Trustee Unified Approval Dashboard

When a task moves into the `READY_FOR_PAYOUT` state, the frontend must render a unified authorization interface specifically for that task's 5 trustees. The frontend must fetch the payout payload data containing both layer vectors.

Endpoint: `GET /api/v1/trustees/payouts/:task_id`

Response Payload:

```
{
  "task_id": "kibera-clean-water-004",
  "vault_status": "READY_FOR_PAYOUT",
  "layer1_onchain_payout": {
    "unsigned_psbt_hex": "7073627466ff...",
    "volunteer_address": "bc1q5d70...",
    "amount_sats": 1000000
  },
  "layer2_lightning_payout": {
    "volunteer_invoice": "lnbc450u1p...",
    "amount_sats": 45000
  }
}
```

5.3 Constructing and Transmitting the Multi-Sig Signature Payload

The frontend must perform two distinct cryptographic operations when a trustee clicks "Approve Payout":

1. **Layer 1 (Hardware Wallet / PSBT):** The frontend communicates with the trustee's hardware device (via WebUSB or QR code scanning) to sign the `unsigned_psbt_hex`. It extracts the generated signature fragment.

2. **Layer 2 (Web-Crypto API / Phone Passkey):** The frontend uses the browser's native subtle crypto framework to sign the raw `volunteer_invoice` string using the private web key generated during onboarding.

Endpoint: `POST /api/v1/trustees/payouts/:task_id/sign`

Payload Sent by Frontend:

```
{
  "trustee_public_key_hex": "03b2241f...",
  "layer1_psbtsignature_fragment": "30440220...",
  "layer2_web_crypto_signature": "e695f2d4739..."
}
```

The backend intercepts this payload, verifies both signature properties, and logs them. The exact second a 3rd unique trustee uploads a valid signature payload, the Go service executes the multi-layered payout instantly.

Frontend Team Architecture Check

The frontend does not need to compute Bitcoin transaction structures. It acts strictly as a secure pass-through layer that reads raw hexadecimal inputs from the backend, pipes them to the user's localized signing keys, and passes the resulting signature fragments back to the API.